THEORY · PYTHON · WEEK 1

Euler's method

The idea

An ODE tells you the slope of the solution at every point:

$$\frac{dy}{dt} = f(t, y), \qquad y(t_0) = y_0$$

Euler's method walks a short step Δt in the direction of the current slope, then works out the slope again at the new point, and repeats:

$$y_{n+1} = y_n + \Delta t\, f(t_n, y_n), \qquad t_{n+1} = t_n + \Delta t$$

Here y_n is the approximation of $y(t_n)$.

For decay, $f = -\lambda N$, so one step is

$$N_{n+1} = N_n - \lambda N_n\, \Delta t$$

Where it comes from

Taylor-expand the exact solution:

$$y(t + \Delta t) = y(t) + \Delta t\, y'(t) + \tfrac{1}{2}\Delta t^2\, y''(t) + \dots$$

Euler keeps the first two terms and drops the rest. Each step therefore makes an error of order Δt^2. This is the *local error*.

Error and order

To reach a fixed time T you take $T/\Delta t$ steps, and the local errors add up: $(T/\Delta t) \cdot \Delta t^2 \sim \Delta t$.

In general a method's *global error* at a fixed time behaves like

$$E \approx C\, \Delta t^{\,p}$$

where p is the **order** of the method. Halving Δt divides the error by 2^p. The ratio of successive errors therefore tells you p, and that ratio is what the last column of your table measures.

Stability (worth knowing)

For decay, each Euler step multiplies N by $(1 - \lambda\Delta t)$:

- If $\lambda\Delta t > 1$, the factor is negative, so N flips sign every step.
- If $\lambda\Delta t > 2$, $|1 - \lambda\Delta t| > 1$, so N grows without bound.

The exact solution never does either, so the step size is limited by stability as well as accuracy. In this exercise $\lambda\Delta t \le 0.28$, which is safe.

Practical tips

- **Number of steps:** use `n = round(t_end / dt)`, not `int(...)`. Floating-point division can land just below a whole number: `int(0.3 / 0.1)` gives `2`, not `3`.
- **Preallocate:** create `N = np.zeros(n + 1)`, then fill it in the loop. Fortran and MATLAB work the same way.